Hybrid Drift Triggering for Industrial Data Streams: A Kafka–River–InfluxDB–Grafana Pipeline

Team Member 1, Team Member 2, Team Member 3, Team Member 4
IOE413 Streaming Data Analytics, Topic 12
Department, Institution, City, India
{email1, email2, email3, email4}@institution.edu

***Abstract*—Machine-learning models deployed on industrial sensor streams silently lose accuracy when the data-generating process changes, a phenomenon known as concept drift. We build an end-to-end streaming pipeline (Apache Kafka, Spark Structured Streaming, the River online-learning library, InfluxDB and Grafana) that monitors a live classifier and raises drift alarms. As the baseline we reproduce the detector benchmark of Sakurai et al. (DDM, EDDM, HDDM-A, HDDM-W) and add ADWIN. Our enhancement is a hybrid trigger that fuses a fast error-rate test (HDDM-A) with a window-mean test (ADWIN) using an OR rule and a refractory period. On 90 synthetic streams (abrupt, gradual, incremental drift) the hybrid attains the highest mean F1 (0.733 versus 0.672 for the best single detector; 0.786 versus 0.723 on 30 further unseen streams) at similar delay and false-alarm rate, but the improvement is not statistically significant under a Friedman/Nemenyi test. Confirmation-style (AND) fusion and EDDM-based hybrids performed worse than single detectors; we report these negative results. In a prequential study on a simulated IIoT stream, drift-triggered LightGBM retraining restores accuracy from 0.59 (static model) to 0.81–0.86. Results use synthetic and simulated data only; validation on IoTID20 and a measured end-to-end Kafka run remain future work.**

***Index Terms*—concept drift, streaming analytics, industrial IoT, ADWIN, HDDM, River, Kafka**

I. Introduction

Industrial systems such as production lines, condition-monitoring networks and industrial intrusion-detection systems produce high-velocity streams whose statistics change with equipment wear, firmware updates, process reconfiguration and new attack behaviour. A classifier trained offline on historical data degrades under such *concept drift* [1], which in predictive maintenance and security has safety and reliability consequences. A batch-only workflow finds the problem late, since accuracy is measured only after labels are aggregated and the model retrained offline. Detection must therefore run online, in bounded memory, and per sample.

Problem statement. Which drift-detection strategy offers the best trade-off between detection accuracy (F1), detection delay, false alarms and computational overhead on industrial streams, and can a simple hybrid of two detectors improve on the best single detector?

Contributions. (i) A working streaming pipeline in which a Kafka topic feeds an online learner with a drift detector whose outputs are written to InfluxDB and shown in Grafana. (ii) A reproduction of the detector-benchmark protocol of Sakurai et al. [2] with a tolerance-window scoring rule and statistical testing. (iii) A hybrid HDDM-A/ADWIN trigger, with ablations against AND-fusion and EDDM-based pairings. (iv) A downstream study of drift-triggered retraining of a gradient-boosted model.

II. Related Work

Base paper 1 (reproduced baseline). Sakurai et al. [2] benchmark the error-rate detectors DDM [5], EDDM [6], HDDM-A and HDDM-W [7] on synthetic binary streams with controlled drift type, length and amplitude, using F1, detection time and detection delay with Friedman/Nemenyi testing. They conclude that no single detector dominates on all axes and that detector choice should depend on the drift scenario. We reproduce this protocol with River [9].

Base paper 2. Yang and Shami [3] propose a multi-stage online analytics framework for IIoT network data (MSANA) that pairs ADWIN [8] and EDDM as drift detectors and, on detection, re-selects features and retrains an online ensemble. It is evaluated on IoTID20 and CICIDS2017.

Base paper 3. Yang and Shami [4] present a lightweight framework that combines LightGBM [10] with an optimized adaptive sliding-window drift detector for resource-constrained IoT devices and shows that drift-triggered retraining is more accurate and cheaper than static or naively retrained models. We follow this "detect, then retrain a lightweight model" design for our downstream study.

Gap. Sakurai et al. evaluate detectors in isolation on synthetic streams; Yang and Shami use detector pairs inside larger pipelines, but the effect of the *fusion rule* between two detectors is not isolated. We study that rule directly, on controlled ground truth and inside a running stream pipeline.

III. System Architecture

Fig. 1 shows the pipeline. A producer replays a simulated IIoT sensor stream (eight sensor channels with a normal/fault label and a known drift schedule) into the Kafka topic `sensors` at a configurable rate. The online scorer, a Python consumer, reads each record, predicts with a River Hoeffding Adaptive Tree, compares the prediction with the label (prequential test-then-train), updates the drift detector with the resulting error bit, and replaces the learner on an alarm. Per-record error, rolling error, alarm and ground-truth drift flags are written to InfluxDB as measurement `drift_stream`, and a provisioned Grafana dashboard plots rolling error, alarms

and true drift onsets with a five-second refresh. In parallel, a Spark Structured Streaming job reads the same topic and computes 10 s windows (sliding 5 s) of per-sensor mean, standard deviation and class rate. In the current implementation this branch writes to the console and does not feed the detector; it demonstrates the windowed feature layer but is not part of the scoring path.

Why these tools. Kafka provides a durable, replayable, ordered log that decouples the sensor source from the consumers, so the same recorded stream can drive the scorer and the Spark job independently and experiments can be repeated. River updates a model and a drift detector one sample at a time in constant memory, which is the property a batch library lacks. Spark Structured Streaming supplies event-time windows with watermarks for feature aggregation that would otherwise be hand-written. InfluxDB stores the resulting time series and Grafana renders them live, giving the monitoring view an operator needs. A single script reading a CSV once would compute the same detector scores but would not exercise transport, windowing, storage or visualization, nor show behaviour on a live feed.

IV. Proposed Enhancement

Error-rate detectors differ in what they are sensitive to: DDM/HDDM-A test a running mean of the error bit against its historical minimum, HDDM-W applies exponentially weighted moving averages, EDDM monitors the distance between consecutive errors, and ADWIN adaptively shrinks a window whenever two sub-windows have significantly different means [8]. We combine two detectors with different mechanisms:

$$\text{alarm}_t = \big(f_t \vee s_t\big) \wedge (t - t_{\text{last}} > R), \tag{1}$$

where f_t is HDDM-A, s_t is ADWIN, t_{last} is the previous emitted alarm and R is a refractory period. The OR rule raises recall, since the two tests fire on different samples, and the refractory period merges the bursts of alarms that follow a single drift, protecting precision. The pairing and $R = 500$ were chosen during exploratory runs on the evaluation seeds 0–29, so results on those seeds are optimistically biased. A later sweep over $R \in \{100, 200, 300, 500, 800\}$ on separate seeds 1000–1019 (Table IV) and a confirmatory benchmark on untouched seeds 2000–2029 (Table V) were run to check this. We also implement a confirmation rule (f and s within $k = 300$ samples of each other) and the pairing HDDM-W/EDDM as ablations. Both detectors are used with River's default parameters.

V. Implementation

The code is Python 3.12 with River 0.26.1, LightGBM 4.7.0, NumPy 2.4.4, SciPy 1.17.1, scikit-learn 1.8.0 and pandas 3.0.2. The streaming infrastructure is configured in `docker-compose.yml` with Apache Kafka 3.7.0 in KRaft mode, InfluxDB 2.7 and Grafana 10.4.2; the Spark job targets Spark 3.5 with the Kafka connector. Kafka clients use `kafka-python` and the InfluxDB writer uses `influxdb-client`.

TABLE I
Key configuration choices.

Item	Setting
Synthetic streams	10 000 samples, 3 drifts (2 500/5 000/7 500), error rate 0.10 rising by 0.12 per drift, width 1 (abrupt) or 500
Tolerance window	$[t,\, t + w + 300)$, capped at next onset
Detectors	River defaults for DDM, EDDM, HDDMA, HDDMW, ADWIN
Hybrid	HDDM-A OR ADWIN, refractory $R = 500$
Simulated IIoT stream	20 000 samples, 8 features, drifts at 5 000, 10 000, 15 000 (gradual, 1 500)
LightGBM	80 trees, learning rate 0.1, 15 leaves, single thread
Retraining	wait 250 samples after alarm, train on latest 500
Prequential batch	25 samples; initial training on first 1 000
River baselines	ARF with 5 trees; Hoeffding Adaptive Tree; seed 1
Kafka replay	200 records/s, topic `sensors`
Spark windows	10 s length, 5 s slide, 10 s watermark

Synthetic drift streams. Following the protocol of [2], each stream is a binary error indicator of length 10 000 with three drifts at positions 2 500, 5 000 and 7 500. At each drift the error probability rises by 0.12 from a base of 0.10. Abrupt drift switches instantly; incremental drift ramps the probability linearly over 500 samples; gradual drift draws each sample from the new concept with a linearly increasing probability over 500 samples. We generate 30 seeds per drift type (90 streams).

Simulated IIoT stream. For the downstream study we generate 20 000 samples with eight features and a binary label from a linear boundary plus an interaction term and noise. The boundary changes abruptly at samples 5 000 and 10 000 and gradually over 1 500 samples from 15 000, so drift is real concept drift.

Downstream learners. The first 1 000 samples train the initial LightGBM model. Evaluation is prequential in micro-batches of 25. For drift-triggered variants the detector receives each error bit; after an alarm the pipeline waits 250 samples and retrains on the latest 500 labelled samples. LightGBM is not an online learner, so this delayed retraining is an explicit design choice. River's Adaptive Random Forest (five trees) and Hoeffding Adaptive Tree serve as true online baselines.

VI. Experimental Setup and Results

Scoring rule. A true drift onset t is detected if the first alarm falls in $[t,\, t + w + 300)$ and before the next onset, where w is the drift width. Every other alarm is a false alarm and an unmatched onset is a miss. We report F1, mean detection delay over detected drifts, false alarms per 1 000 samples, per-sample detector time and peak allocated memory (`tracemalloc`, one stream). Detectors are compared with the Friedman test on per-stream F1 ($N = 90$ blocks, $k = 9$ detectors) and the Nemenyi critical difference [11].

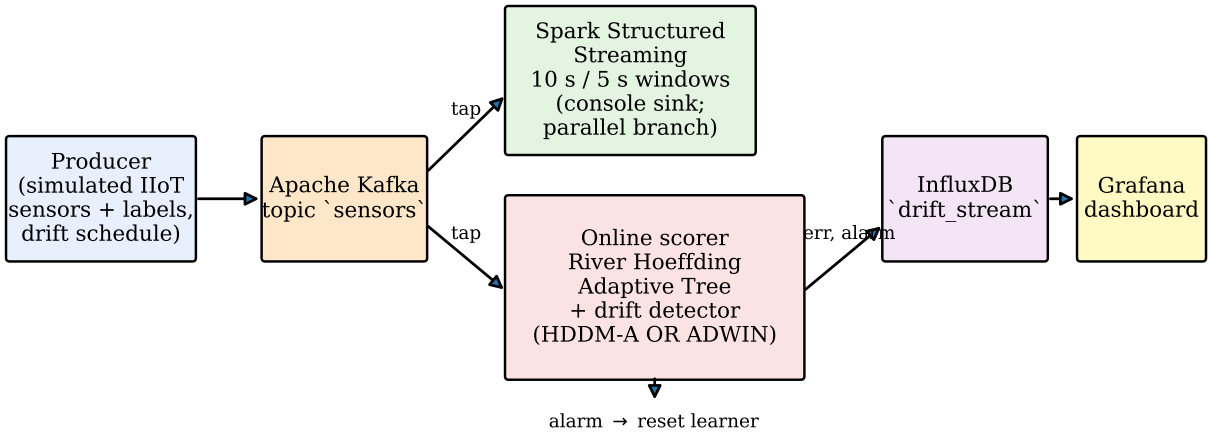


Fig. 1. Pipeline architecture and data flow. The Spark branch is a parallel consumer of the Kafka topic.

TABLE II
OVERALL DETECTOR RESULTS (90 STREAMS). FA = FALSE ALARMS PER 1 000 SAMPLES; DELAY IN SAMPLES; TIME IN μS PER SAMPLE; MEM = PEAK KB.

Detector	F1	Delay	FA	μs	kB	Rank
Hybrid OR(A,ADWIN)	0.733	445	0.097	7.4	4.3	2.23
HDDM-A	0.672	469	0.110	6.5	5.1	2.89
ADWIN	0.611	488	0.117	0.4	0.8	3.17
HDDM-W	0.464	317	0.357	4.9	6.1	4.68
DDM	0.418	451	0.278	0.8	0.9	5.16
AND(HDDM-W,EDDM)	0.417	346	0.390	6.1	5.8	5.27
AND(HDDM-W,ADWIN)	0.370	536	0.080	5.9	5.5	5.98
OR(HDDM-W,EDDM)	0.223	182	2.087	6.1	6.7	7.27
EDDM	0.134	180	3.784	0.6	3.2	8.35

TABLE III
F1 / MEAN DELAY (SAMPLES) BY DRIFT TYPE.

Detector	Abrupt	Gradual	Incremental
EDDM	0.124 / 77	0.153 / 276	0.125 / 182
DDM	0.231 / 188	0.524 / 545	0.498 / 556
HDDM-W	0.404 / 119	0.485 / 411	0.503 / 408
ADWIN	0.422 / 217	0.733 / 615	0.678 / 596
HDDM-A	0.505 / 188	0.792 / 622	0.719 / 566
Hybrid	0.613 / 193	0.819 / 583	0.767 / 551

TABLE IV
REFRACTORY-PERIOD SWEEP OF THE HYBRID ON HELD-OUT SEEDS 1000–1019 (60 STREAMS).

R	F1	Delay	FA / 1 000
100	0.634	424	0.215
200	0.702	424	0.145
300	0.722	424	0.125
500	0.755	424	0.093
800	0.775	424	0.075

Detector benchmark. Table II and Fig. 2 give the results. The Friedman test rejects equal performance ($\chi^2 = 410.1$, $p < 10^{-80}$); the Nemenyi critical difference is 1.27 rank units. The hybrid has the best mean rank (2.23) and F1 (0.733), ahead of HDDM-A (2.89, 0.672) and ADWIN (3.17, 0.611). The rank differences from HDDM-A (0.66) and ADWIN (0.94) are below the critical difference, so the hybrid is *not* statistically distinguishable from these two. Differences to HDDM-W, DDM, EDDM and all ablations exceed it.

Per drift type (Table III), the hybrid's advantage over HDDM-A is largest for abrupt drift (F1 0.613 vs. 0.505) with essentially unchanged delay (193 vs. 188), and smaller for incremental (0.767 vs. 0.719) and gradual drift (0.819 vs. 0.792). Delays for gradual and incremental drift are several hundred samples for every detector except EDDM, which reacts fastest (76–276 samples) but at a cost of 3.4–4.1 false alarms per 1 000 samples and F1 between 0.12 and 0.15.

Tuning and confirmation. Table IV shows the refractory sweep on seeds 1000–1019: F1 rises steadily with R (0.634 at 100 to 0.775 at 800) while the delay stays near 424 samples, so $R = 500$ is not the optimum of the range and we did not select it from this sweep. Table V repeats the full benchmark on seeds 2000–2029, which were never used for any design decision. The ordering is the same: the hybrid is first (F1 0.786, rank 2.14), HDDM-A second (0.723, 2.73), ADWIN third (0.641, 3.27), and the hybrid–HDDM-A rank gap (0.59) is again below the critical difference of 1.27 (Friedman $\chi^2 = 426.1$).

Negative results. Confirmation (AND) fusion lowered F1 for both pairings tested (0.370 and 0.417), because requiring agreement between a fast and a slow detector drops recall (0.23–0.66) and lengthens delay while the false-alarm rate barely improves. Pairing HDDM-W with EDDM inherited EDDM's high false-alarm rate under an OR rule and was worse than either HDDM-W alone or the pairing with ADWIN.

Downstream adaptation. Table VI and Fig. 3 show prequential accuracy on the simulated IIoT stream (one stream, seed 0). A static LightGBM falls to 0.586 accuracy overall

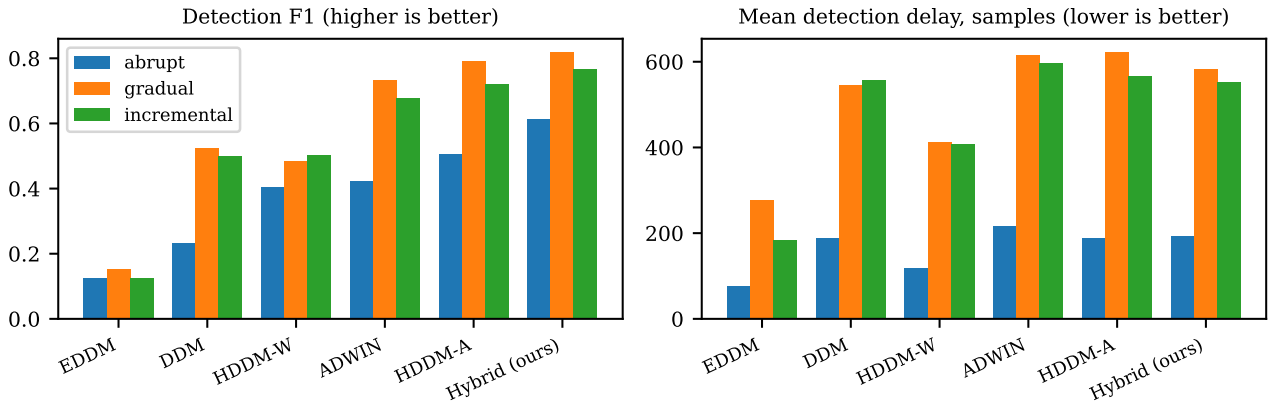


Fig. 2. Detection F1 and mean detection delay by drift type for the single detectors and the proposed hybrid.

TABLE V
CONFIRMATORY BENCHMARK ON UNSEEN SEEDS 2000–2029 (90 STREAMS).

Detector	F1	Delay	FA	Rank
Hybrid OR(HDDM-A,ADWIN)	0.786	449	0.081	2.14
HDDM-A	0.723	464	0.102	2.73
ADWIN	0.641	480	0.108	3.27
DDM	0.480	448	0.248	4.79
HDDM-W	0.456	301	0.353	4.83
AND(HDDM-W,EDDM)	0.417	358	0.373	5.42
AND(HDDM-W,ADWIN)	0.332	521	0.088	6.24
OR(HDDM-W,EDDM)	0.235	212	1.944	7.20
EDDM	0.147	226	3.416	8.38

TABLE VI
PREQUENTIAL RESULTS ON THE SIMULATED IIOT STREAM. ACC = OVERALL ACCURACY; ACC_1 = ACCURACY AFTER FIRST DRIFT; MS = MS PER SAMPLE.

Strategy	Acc	Acc_1	Alarms	Retr.	F1_d	ms
Static LightGBM	0.586	0.497	0	0	–	0.03
+ DDM	0.830	0.811	9	6	0.50	0.04
+ HDDM-A	0.819	0.792	4	4	0.86	0.04
+ ADWIN	0.863	0.847	7	7	0.60	0.04
+ EDDM	0.830	0.810	37	27	0.15	0.07
+ Hybrid (ours)	0.811	0.781	4	4	0.86	0.05
River ARF	0.840	0.837	–	–	–	0.73
River HAT	0.874	0.867	–	–	–	0.15

(0.497 after the first drift). Every drift-triggered variant recovers to between 0.81 and 0.86. The hybrid trigger has the best detection F1 (0.857, tied with HDDM-A) and a delay of 220 samples versus 268 for HDDM-A, but its overall accuracy (0.811) is lower than ADWIN's (0.863) and River's Hoeffding Adaptive Tree (0.874). EDDM triggers 37 alarms and 27 retrains for three true drifts. The online Hoeffding Adaptive Tree needs no external detector and was the most accurate strategy here.

Live pipeline behaviour. With the hybrid detector and a River Hoeffding Adaptive Tree replaced on each alarm, the in-process runner raised alarms at samples 5 025, 10 044 and 15 519 (delays 25, 44 and 519 samples) plus seven other alarms (351, 2 916, 4 223, 8 991, 10 641, 17 663 and 18 207), i.e. every true drift was caught but with many extra alarms.

VII. DISCUSSION AND INFERENCES

What the results say. Detectors that watch a running mean (HDDM-A, ADWIN) were the most reliable on our synthetic streams; sensitive detectors (EDDM, HDDM-W) reacted faster but produced many more false alarms, so under an F1 that penalizes every extra alarm they rank lower. The OR-fusion of two mean-based detectors with different mechanisms increased recall (0.76 versus 0.69 for HDDM-A alone, from held-out tuning) while the refractory period kept false alarms at 0.097 per 1 000 samples. The hybrid's gain is modest and, under the Nemenyi test, not significant against HDDM-A or ADWIN. It costs about one microsecond more per sample than HDDM-A, negligible next to any Kafka hop.

Relation to base paper 1. Sakurai et al. report EDDM as the most accurate detector overall and HDDM-W as the best trade-off. Our results differ: EDDM has the highest recall and shortest delay but the lowest F1. We attribute this mainly to our scoring rule (all extra alarms are false alarms), our one-sided error-rate increases, and different stream parameters; we did not tune any baseline detector. We do not claim to have reproduced their ranking.

Downstream. Drift detection is worth doing: static LightGBM lost about 40 accuracy points. However, better detection F1 did not translate into better accuracy in our single run, since accuracy also depends on when retraining happens and how much post-drift data is available (250-sample wait, 500-sample window), and on false-alarm retrains that happen to be harmless. An online tree with built-in drift handling outperformed all retrain-on-alarm variants, so a detector-plus-retrain design must be justified by other benefits (model choice, cost, interpretability) rather than accuracy alone.

Threats to validity. (1) All detector results use synthetic Bernoulli error streams; real classifier errors are autocorrelated. (2) The fusion pair and R were chosen on the evaluation

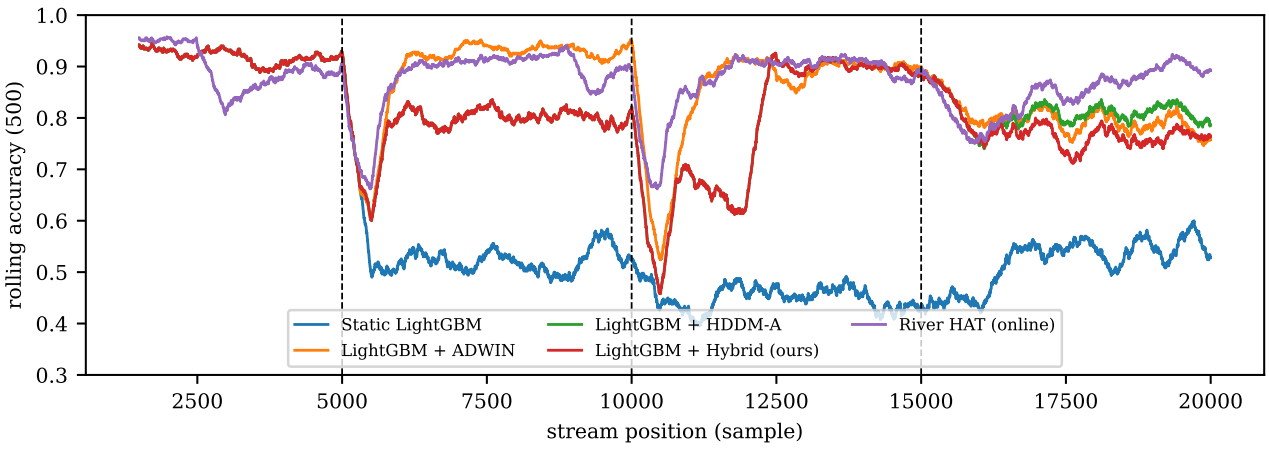


Fig. 3. Rolling prequential accuracy (window 500). Dashed lines mark true drift onsets.

seeds 0–29, and only the hybrid was tuned while the baselines run at River defaults, which favors the hybrid; the unseen-seed run (Table V) shows the same ordering but does not remove the untuned-baseline advantage. (3) The 300-sample margin in the tolerance window and the choice of drift amplitude are ours. (4) Mean delay is computed over detected drifts only, so detectors that miss drifts have optimistic delay. (5) The downstream study uses one stream and one seed. (6) IoTID20 and CICIDS2017 were not used, so no claim is made about real network traffic; real datasets carry no drift labels and would need injected drift. (7) ADWIN is two-sided and can fire when a freshly reset learner *improves*, which likely explains some of the extra alarms in the live run. (8) The Spark branch is not on the scoring path, and end-to-end Kafka-to-Grafana latency and throughput were not measured.

Deployment considerations. The detectors add microseconds per sample and a few kilobytes of state (Table II), so on the detector side the cost is dominated by the classifier and by transport, not by drift detection. Two practical points follow from the live run. First, a detector fed by a learner that has just been reset sees a falling error rate, and two-sided tests such as ADWIN can treat improvement as change; guarding the detector for a warm-up period or using a one-sided variant would remove this class of alarms. Second, in a plant an alarm should trigger a review or a controlled retrain rather than an unattended model swap, so the alarm precision reported here matters more than the delay.

VIII. Reproducibility

The repository provides the commands `python -m drift.benchmark --seeds 30` (exploratory seeds), `python -m drift.benchmark --seeds 30 --start 2000 --out results_heldout` (confirmatory seeds), `python -m drift.tune_hybrid` (refractory sweep), `python -m drift.adaptive` (downstream study) and `python -m streaming.run_local` (in-process pipeline), plus `docker compose up -d`, `python -m streaming.consumer` and `python -m streaming.producer` for the Kafka–InfluxDB–Grafana path. All random seeds are fixed. Wall-clock timings depend on the machine and vary between runs.

IX. Conclusion and Future Work

We built a streaming drift-monitoring pipeline and compared error-rate and window-based detectors on controlled synthetic drift. A hybrid of HDDM-A and ADWIN with a refractory period gave the best mean F1 and rank, though not significantly better than its components, while confirmation-based hybrids and EDDM-based pairings failed. Drift-triggered retraining recovers most of the accuracy a static model loses. Future work: evaluate on IoTID20/CICIDS2017 with injected drift, tune all detectors on equal footing, use one-sided ADWIN or a warm-up guard after learner resets, route Spark window features into the detector, and measure end-to-end latency, throughput and drift-recovery time on the Kafka pipeline.

References

[1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.

[2] G. Y. Sakurai, J. F. Lopes, B. B. Zarpelão, and S. Barbon Junior, "Benchmarking change detector algorithms from different concept drift perspectives," *Future Internet*, vol. 15, no. 5, p. 169, 2023.

[3] L. Yang and A. Shami, "A multi-stage automated online network data stream analytics framework for IIoT systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 2107–2116, 2023.

[4] L. Yang and A. Shami, "A lightweight concept drift detection and adaptation framework for IoT data streams," *IEEE Internet Things Mag.*, vol. 4, no. 2, pp. 96–101, 2021.

[5] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. SBIA*, LNCS 3171, 2004, pp. 286–295.

[6] M. Baena-García *et al.*, "Early drift detection method," in *Proc. 4th Int. Workshop Knowledge Discovery from Data Streams*, 2006.

[7] I. Frías-Blanco *et al.*, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 3, pp. 810–823, 2015.

[8] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, 2007, pp. 443–448.

[9] J. Montiel *et al.*, "River: machine learning for streaming data in Python," *J. Mach. Learn. Res.*, vol. 22, no. 110, pp. 1–8, 2021.
[10] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. NeurIPS*, 2017, pp. 3146–3154.
[11] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.